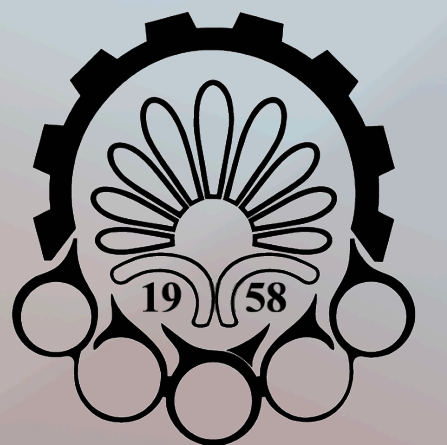


Radial Basis Function Neural Networks (RBFNN)

Computation Intelligence and its Application in Mechatronics

Winter 2025

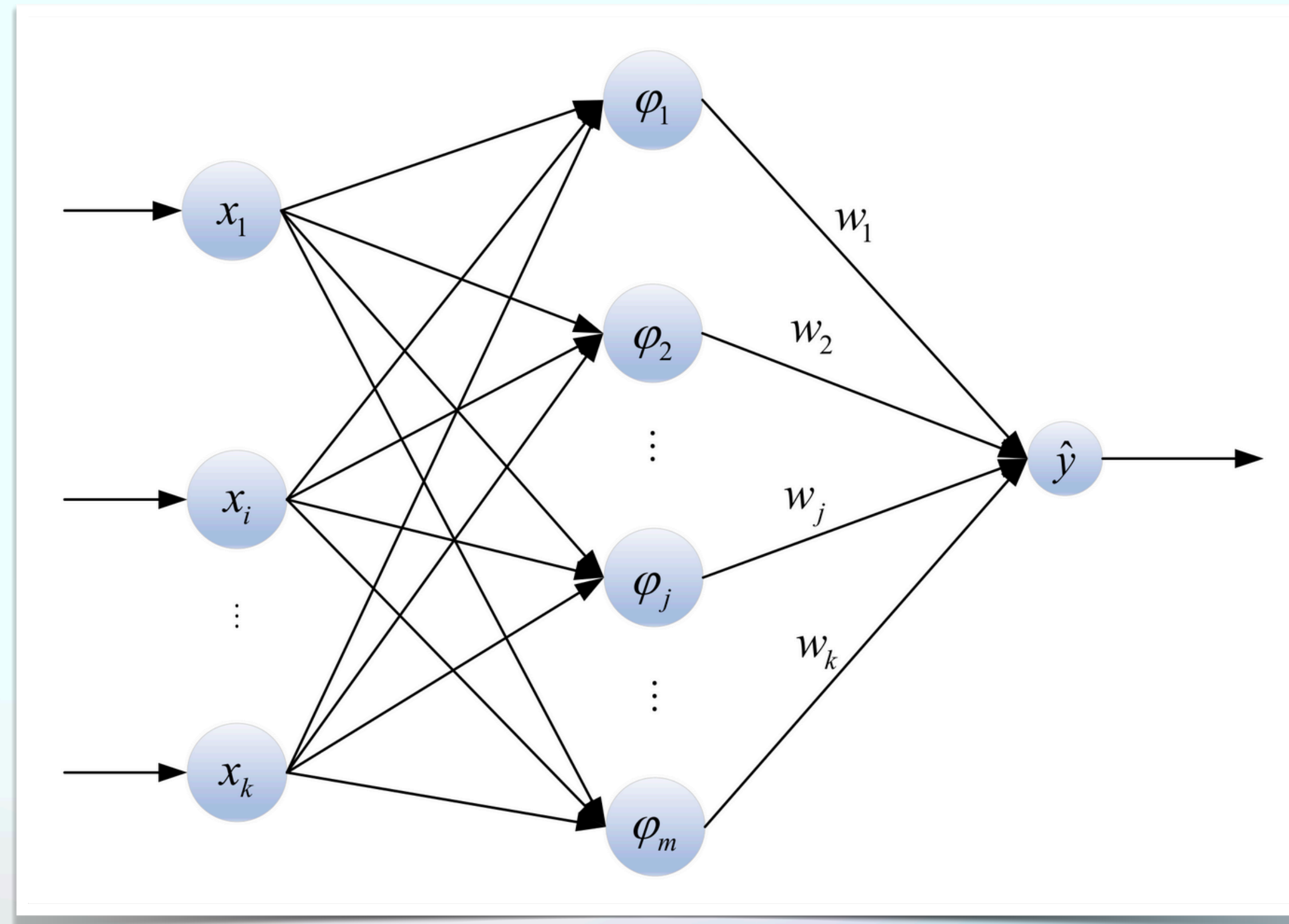


Amirkabir University of Technology
(Tehran Polytechnic)

RBFNN

Structure

- Input Layer
- Hidden Layer (applies RBFs).
- Output Layer



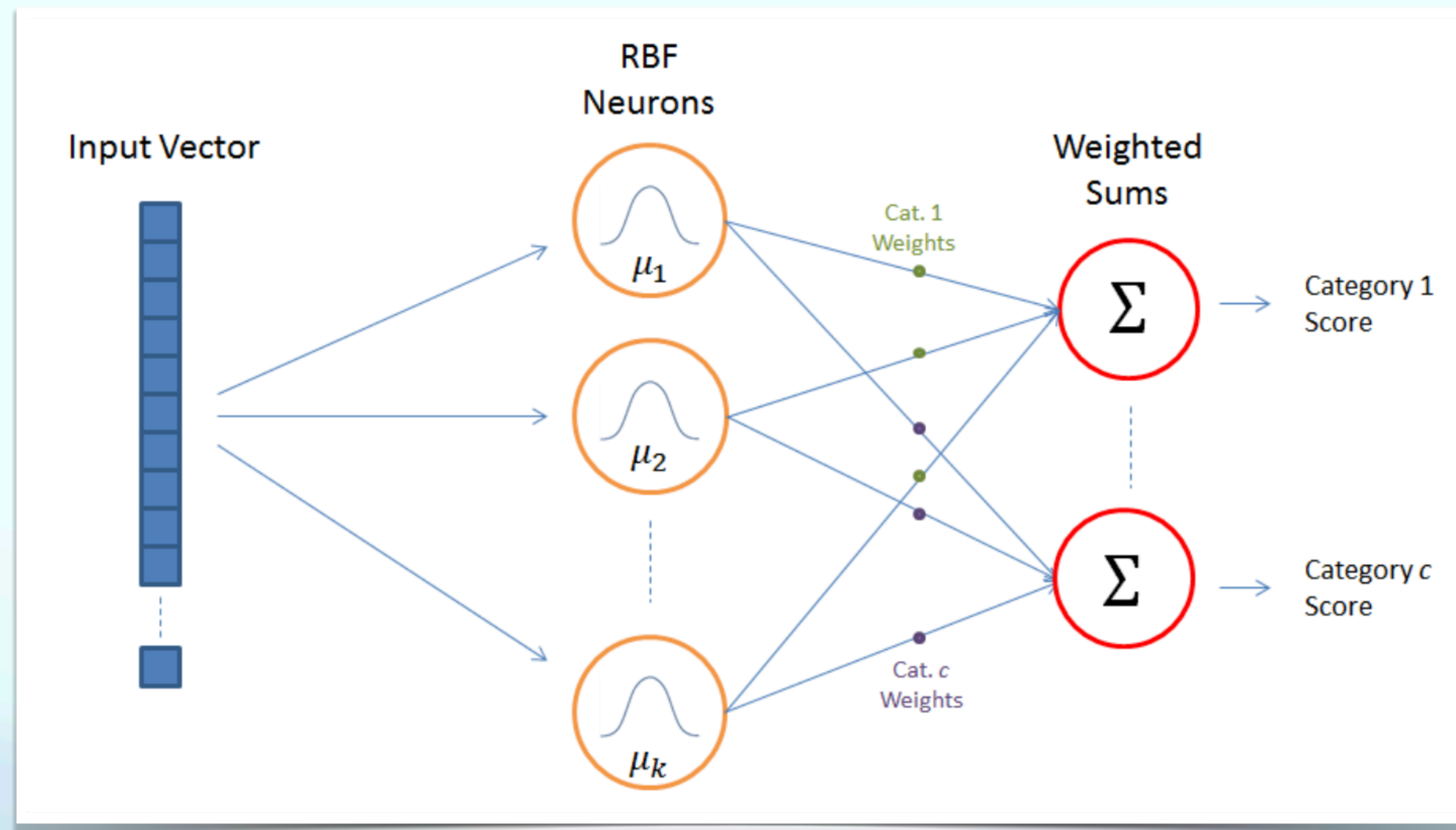
$$\hat{y} = \sum_{i=1}^M w_i \phi_i(x)$$

The Radial Basis Function

The most common RBF is the Gaussian function:

$$\phi(x) = \exp\left(-\frac{\|x - c\|^2}{2\sigma^2}\right)$$

Gaussian RBFNN



Training RBFNNs

Unsupervised Learning (Center Selection + Least Squares Weights):

Find centers using clustering (e.g., K-Means).

Compute weights using Least Squares.

Supervised Learning (Gradient Descent on Centers & Weights):

Learn centers and weights jointly using backpropagation.

Unsupervised Training Approach

Step 1: Select Centers Using Clustering

- Apply K-Means clustering to find cluster centers from input data.
- Set each cluster centroid as an RBF center.

Step 2: Compute Width (σ)

- Set σ based on cluster spread: $\sigma = \frac{d_{\max}}{\sqrt{2M}}$

Where $d_{\max}$ is the max distance between centers, and M is the number of neurons.

Supervised Training Approach

Use Gradient Descent / Backpropagation to update:

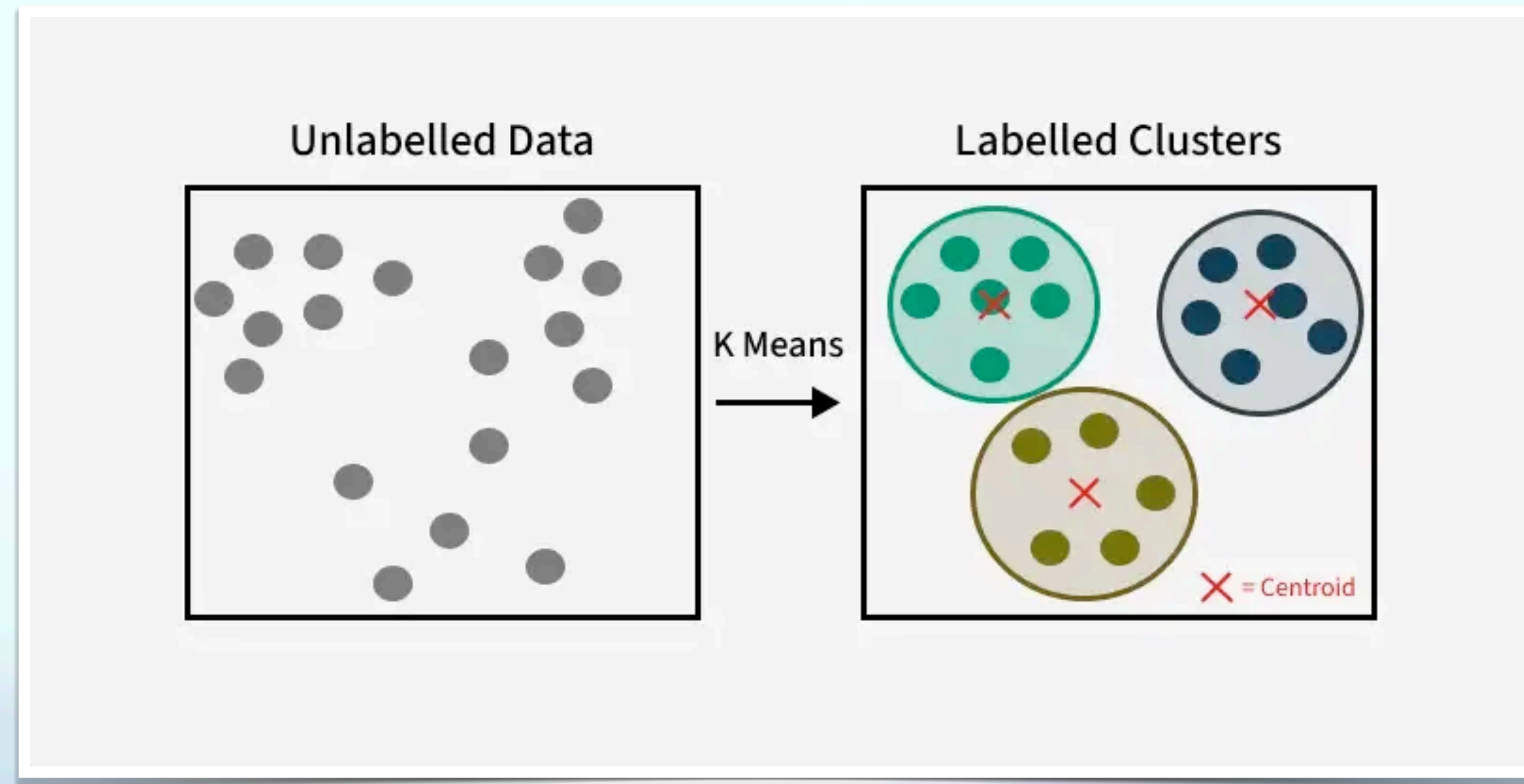
- Centers c_i
- Widths σ_i
- Weights w_i

Loss function:

$$L = \sum_{i=1}^N \left(y_i - \sum_{j=1}^M w_j \phi_j(x_i) \right)^2$$

Appendix

K-Means Algorithm



Appendix

K-Means Algorithm

1. Choose the number of clusters (K)
2. Initialize K cluster centroids randomly.
3. Assign each data point to the nearest centroid based on Euclidean distance:

$$\text{distance} = ||x_i - c_k||^2$$

Where x_i is a data point and c_k is a cluster centroid.

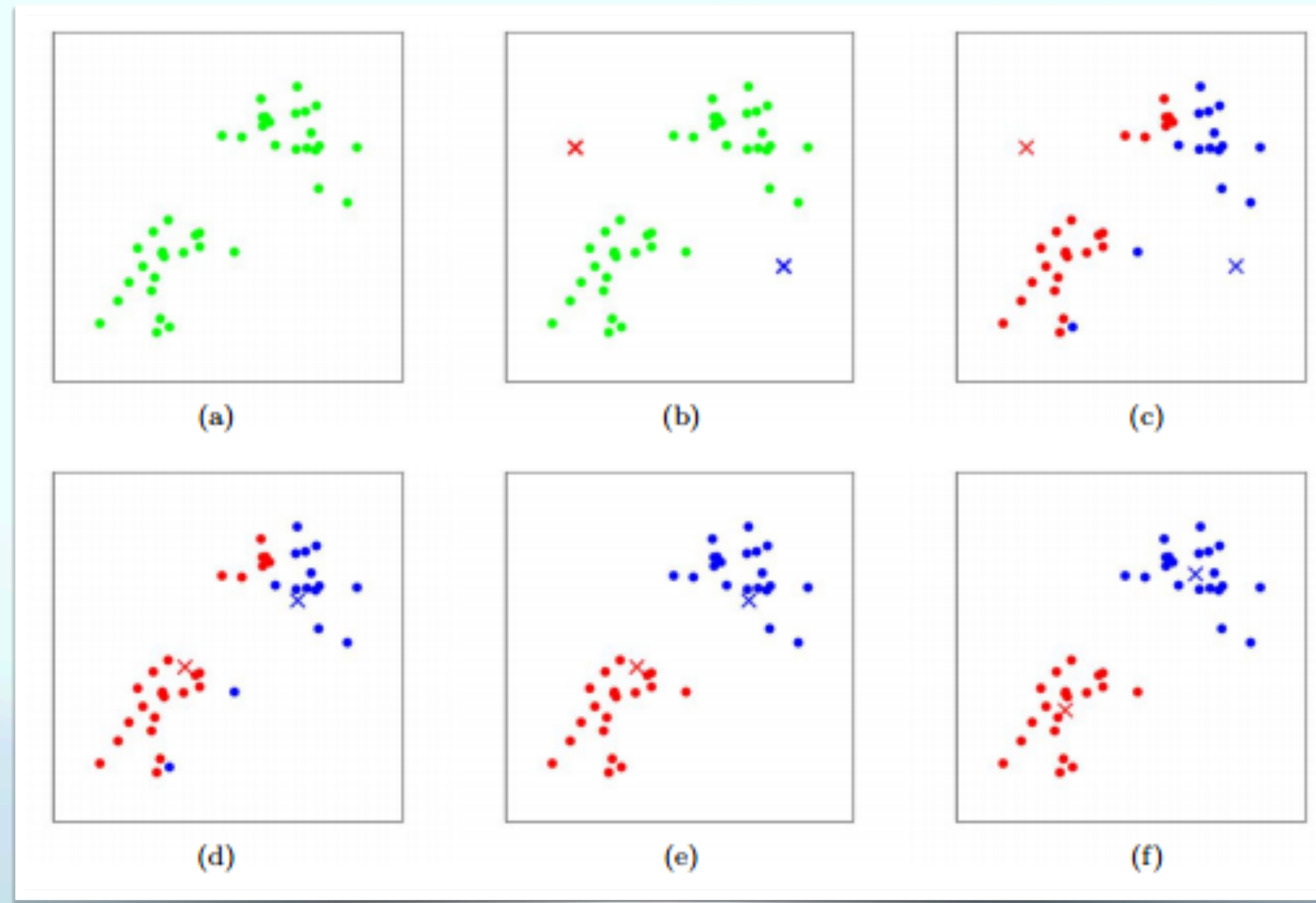
4. Recompute the centroids by taking the mean of all points assigned to each cluster:

$$c_k = \frac{1}{N_k} \sum_{x_i \in C_j} x_i$$

5. Repeat steps 3-4 until convergence (centroids do not change or changes are minimal).

Appendix

K-Means Algorithm



Appendix

K-Means Algorithm

- Implementation:

```
from sklearn.cluster import KMeans  
kmeans = KMeans(n_clusters= number_of_clusters)  
kmeans.fit(X)
```